

UC Berkeley
Teaching Professor
Dan Garcia

CS61C

Great Ideas in Computer Architecture
(a.k.a. Machine Structures)

Dependability (Parity, ECC, & RAID)

BY NC SA

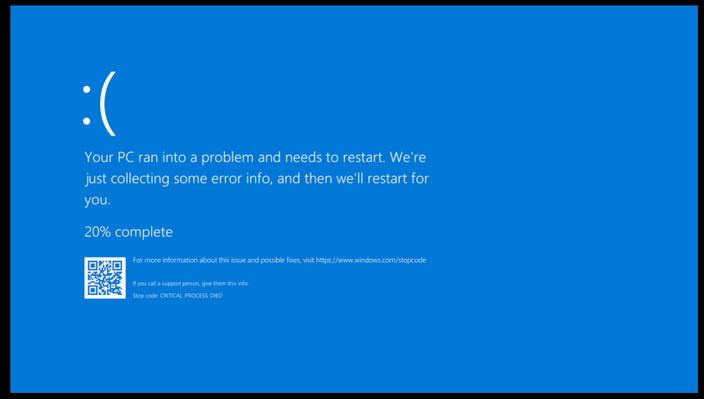
6 Great Ideas in Computer Architecture

1. Abstraction (Layers of Representation/Interpretation)
2. Moore's Law
3. Principle of Locality/Memory Hierarchy
4. Parallelism
5. Performance Measurement & Improvement
6. Dependability via Redundancy

cc BY NC SA

Computers Fail…

- May fail transiently…



- …or permanently

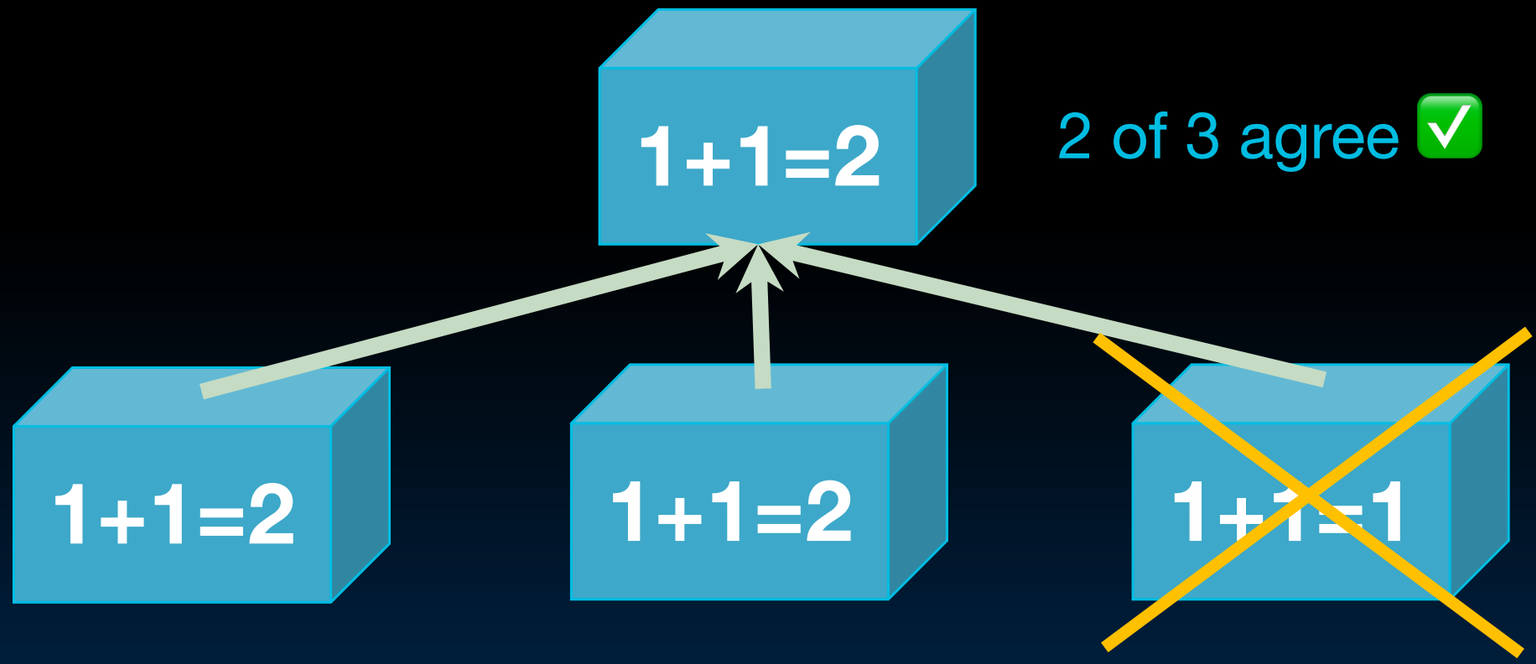


We will discuss hardware failures and methods to mitigate them

CC BY NC SA

Great Idea #6: Dependability via Redundancy

- **Design with redundancy so that a failing piece doesn't make the whole system fail.**



Increasing transistor density reduces the cost of redundancy

CC BY NC SA

Great Idea #6: Dependability via Redundancy

- **Applies to everything from datacenters to storage to memory to instructors!**



Redundant datacenters so that can lose 1 datacenter but Internet service stays online;



Redundant disks so that can lose 1 disk but not lose data (Redundant Arrays of Independent Disks/RAID);

Redundant memory bits so that can lose 1 bit but no data (Error Correcting Code/ECC Memory).



CC BY NC SA

Dependability Metrics

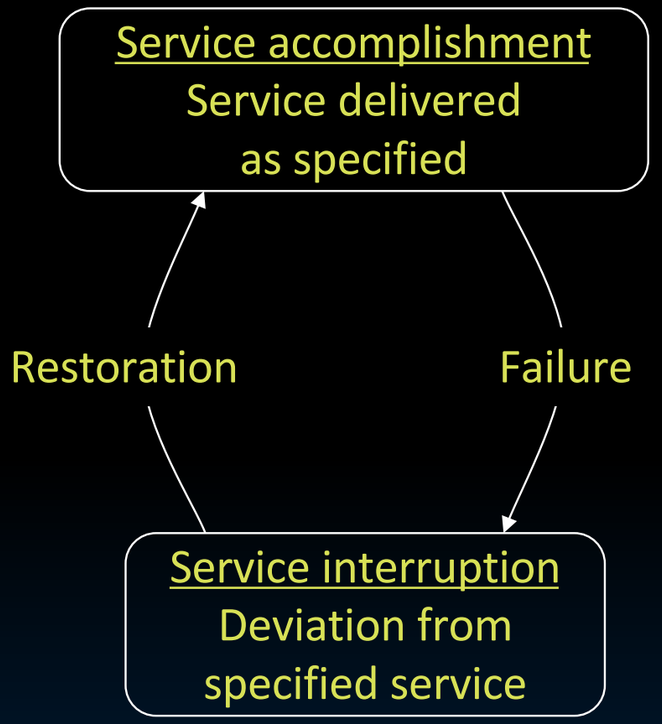
- Dependability Metrics
- Error Detection
- Error Detection and Correction
- Error Correcting Code (ECC) Examples
- Redundancy with RAID

CC BY NC SA

Dependability

- **Fault: failure of a component**

  May or may not lead to system failure



cc BY NC SA

Dependability via Redundancy: Time vs. Space

- **Spatial Redundancy** – replicated data or check information or hardware to handle hard and soft (transient) failures
- **Temporal Redundancy** – redundancy in time (retry) to handle soft (transient) failures

CC BY NC SA

Dependability Measures

- **Reliability: Mean Time To Failure (MTTF) in hrs … want** ⬆️
- **Service interruption: Mean Time To Repair (MTTR) in hrs … want** ⬇️
- **Mean time between failures (MTBF)**
  - MTBF = MTTF + MTTR
- **Availability = MTTF / (MTTF + MTTR) as %**
- **Improving Availability**
  - Increase MTTF: More reliable hardware/software + Fault Tolerance
  - Reduce MTTR: improved tools and processes for diagnosis and repair

CC BY NC SA

Availability Measures

- **Since hope rarely down, shorthand is "number of 9s of availability per year"**
  - 1 nine: 90% ➔ 36 days of repair/year
  - 2 nines: 99% ➔ 3.6 days of repair/year
  - 3 nines: 99.9% ➔ 526 minutes of repair/year
  - 4 nines: 99.99% ➔ 53 minutes of repair/year
  - 5 nines: 99.999% ➔ 5 minutes of repair/year

cc BY NC SA

Dependability Design Principle

- **Dependability Design Principle: No single points of failure**
  - "Chain is only as strong as its weakest link"
- **Dependability corollary of Amdahl's Law**
  - Doesn't matter how dependable you make one portion of system
  - Dependability limited by part you do not improve

CC BY NC SA

Error Detection

- Dependability Metrics
- Error Detection
- Error Detection and Correction
- Error Correcting Code (ECC) Examples
- Redundancy with RAID

cc BY NC SA

Error Detection/Correction Codes

- **Memory systems generate errors (accidentally flipped bits)**
  - DRAMs store very little charge per bit
  - *"Soft" errors* occur occasionally when cells are struck by alpha particles or other environmental upsets
  - *"Hard" errors"* can occur when chips permanently fail
  - Problem gets worse as memories get denser and larger
- **Memories protected against soft errors with** EDC/ECC
- **Extra bits are added to each data-word**
  - Used to detect and/or correct faults in the memory system
  - Each data word value mapped to unique *code word*
  - A fault changes valid code word to invalid one, which can be detected

CC BY NC SA

Block Code Principles

- Hamming distance = **difference in # of bits**
- p = 0**1**1**0**11, q = 0**0**1**1**11, Ham. distance (p,q) = 2
- p = 011011,
  q = 110001,
  distance (p,q) = ?
- Can think of extra bits as creating a code with the data
- What if minimum distance between codewords is 2 and get a 1-bit error?

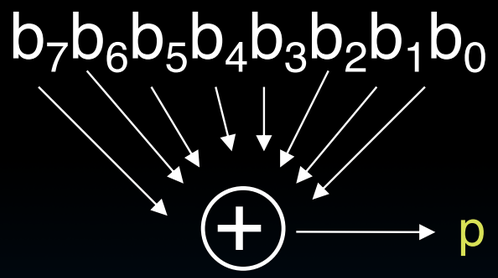


Richard Hamming, 1915-98
Turing Award Winner

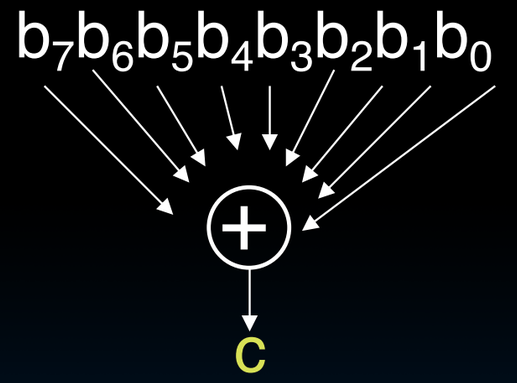
cc BY NC SA

Parity: Simple Error-Detection Coding

- Each data value, before it is written to memory is "tagged" with an extra bit to force the stored word to have *even parity*, via XOR:



- Each word, as it is read from memory is "checked" by finding its parity (including the parity bit).



- A non-zero parity check indicates an error occurred:
  - 2 errors (on different bits) are not detected
  - Nor any even number of errors, just odd numbers of errors are detected

CC BY NC SA

Parity Example

- Data 0101 0101
- 4 ones, even parity now
- Write to memory:
  0101 0101 0
  to keep parity even
- Data 0101 0111
- 5 ones, odd parity now
- Write to memory:
  0101 0111 1
  to make parity even

- Read from memory
  0101 0101 0
- 4 ones → even parity, so no error (or even # of errors)
- Read from memory
  1101 0101 0
- 5 ones → odd parity,
  so error
- What if error is in parity bit?

cc BY NC SA

Error Detection and Correction

- Dependability Metrics
- Error Detection
- Error Detection and Correction
- Error Correcting Code (ECC) Examples
- Redundancy with RAID

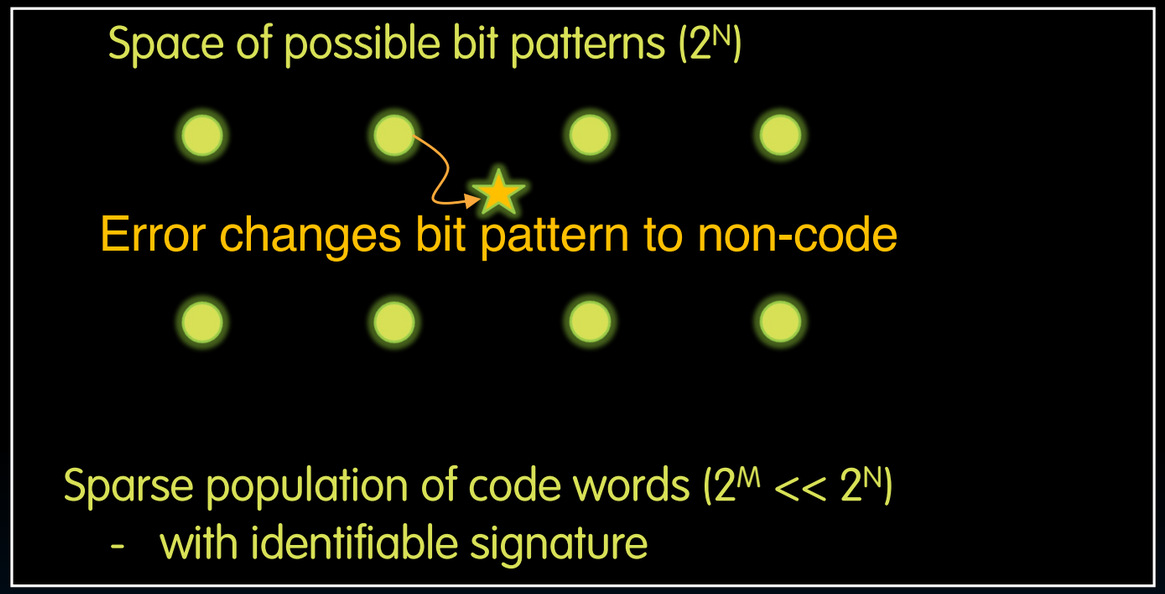
CC BY NC SA

Suppose Want to Correct One Error?

- **Hamming came up with simple-to-understand mapping to allow Error Correction at minimum distance of three**
  - Single error correction, double error detection
- **Called "Hamming ECC," or Hamming Error Correction Code**
  - Worked weekends on relay computer with unreliable card reader, frustrated with manual restarting
  - Got interested in error correction; published 1950
  - R. W. Hamming, "Error Detecting and Correcting Codes," *The Bell System Technical Journal*, Vol. XXVI, No 2 (April 1950) pp 147-160.

BY NC SA

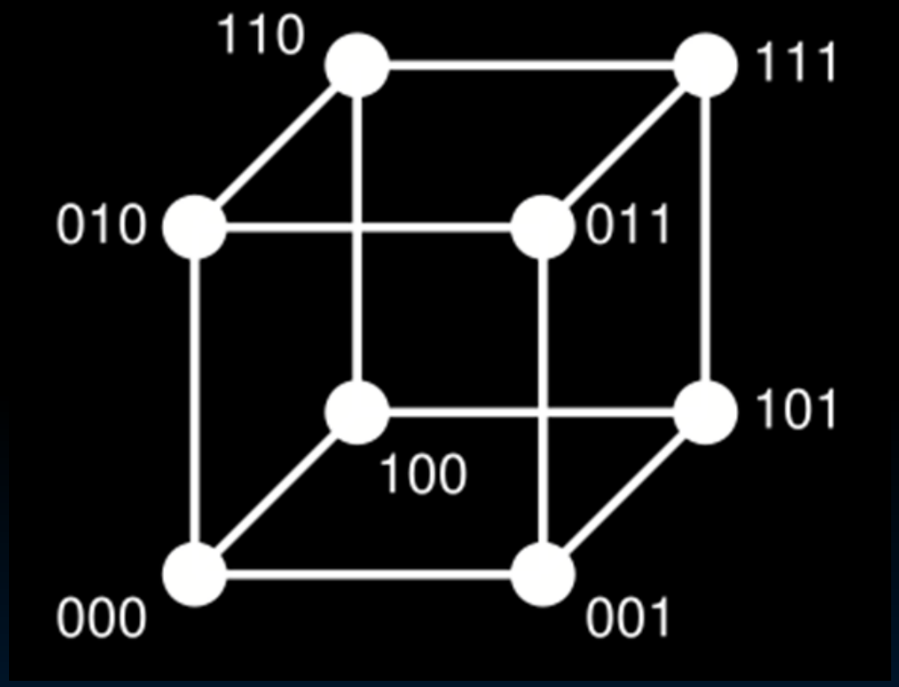
Detecting/Correcting Code Concept



- **Detection**: bit pattern fails codeword check
- **Correction**: map to nearest valid code word

cc BY NC SA

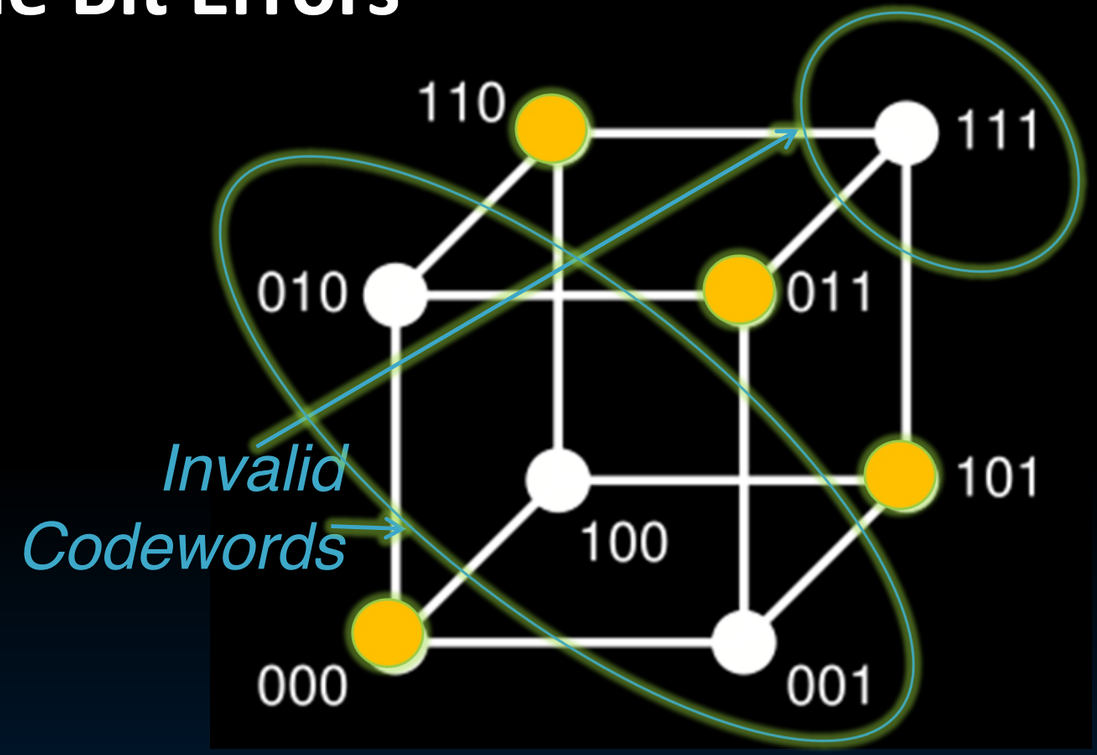
Hamming distance: Eight Code Words



cc BY NC SA

Hamming Distance 2: Detection

- **Detect Single Bit Errors**

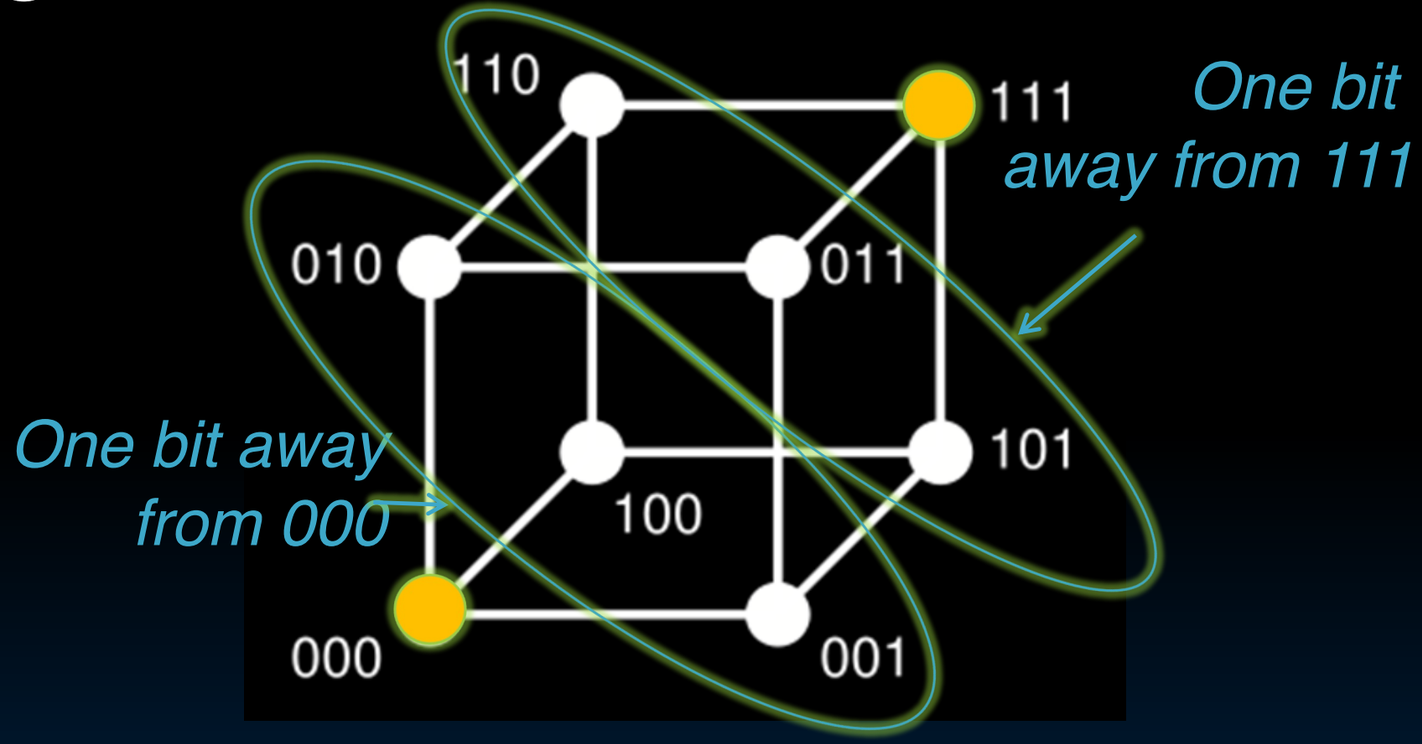


- **No 1-bit error goes to another valid codeword**
- **½ codewords are valid**

cc BY NC SA

Hamming Distance 3: Correction

- **Correct Single Bit Errors**



- **1-bit errors near valid codewords**
- **½ codewords are valid**

BY NC SA

Error Correcting Code (ECC) Example

- Dependability Metrics
- Error Detection
- Error Detection and Correction
- Error Correcting Code (ECC) Examples
- Redundancy with RAID

CC BY NC SA

Hamming ECC

- **The following example is of a Hamming Error Correcting Code (ECC).**
- **Interleave data and parity bits**
- **Place parity bits at binary positions 1, 10, 100, etc .**

  p1 covers all positions with LSB = 1

  p2 covers all positions with next to LSB = 1, etc

  Can continue indefinitely

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit coverage	p1																					...
	p2																					
	p4																					
	p8																					
	p16																					

Kao

Hamming ECC Encoding (1/3)

Set parity bits to create even parity for each group

- **A byte of data: `10011010`**
- **Create the coded word, leaving spaces for the parity bits:**
- `_ _ 1 _ 0 0 1 _ 1 0 1 0`
  `1 2 3 4 5 6 7 8 9 a b c` – bit position

- **Calculate the parity bits**

CC BY NC SA

Hamming ECC Encoding (2/3)

- Position 1 checks bits 1,3,5,7,9,11:
  ? _ 1 _ 0 0 1 _ 1 0 1 0. set position 1 to a _:
- Position 2 checks bits 2,3,6,7,10,11:
  0 ? 1 _ 0 0 1 _ 1 0 1 0. set position 2 to a _:
- Position 4 checks bits 4,5,6,7,12:
  0 1 1 ? 0 0 1 _ 1 0 1 0. set position 4 to a _:
- Position 8 checks bits 8,9,10,11,12:
  0 1 1 1 0 0 1 ? 1 0 1 0. set position 8 to a _:

CC BY NC SA

Hamming ECC Encoding (3/3)

- **Final code word:** 01101001010
- **Data word:** 1 001 1010

CC BY NC SA

Hamming ECC Decoding (1/4)

- Suppose receive
  011100101110

0 1 1 1 0 0 1 0 1 1 1 0

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit cove-rage	p1																					...
	p2																					
	p4																					
	p8																					
	p16																					

rcia, Kao
NC SA

Hamming ECC Decoding (2/4) – Error Check

- **Suppose receive**

```
0 1 1 1 0 0 1 0 1 1 1 0
0 1 0 1 1 1                ✅
  1 1   0 1   1 1          ❌-Parity 2 in error
    1 0 0 1       0        ✅
            0 1 1 1 0      ❌-Parity 8 in error
```

- Implies position 8+2=10 is in error

```
0 1 1 1 0 0 1 0 1 1 1 0
```


CC BY NC SA

Hamming ECC Decoding (2/4) – Error Correct

- **Flip the incorrect bit …**

  0 1 1 1 0 0 1 0 1 0 1 0

CC BY NC SA

Hamming ECC Decoding (2/4) – Error Correct

- Suppose receive

```
01110010 1 0 10
0 1 0 1 1 1   ✅
 11   01   01  ✅
   1001     0  ✅
        01010  ✅
```


cc BY NC SA

What if More Than 2-Bit Errors?

- **Use double-error correction, triple-error detection (DECTED)**
- **Network transmissions, disks, distributed storage common failure mode is bursts of bit errors, not just one or two bit errors**
  - Contiguous sequence of *B* bits in which first, last and any number of intermediate bits are in error
  - Caused by impulse noise or by fading in wireless
  - Effect is greater at higher data rates
- **Solve with Cyclic Redundancy Check (CRC), interleaving or other more advanced codes**

cc BY NC SA

Redundancy with RAID

- Dependability Metrics
- Error Detection
- Error Detection and Correction
- Error Correcting Code (ECC) Examples
- Redundancy with RAID

CC BY NC SA

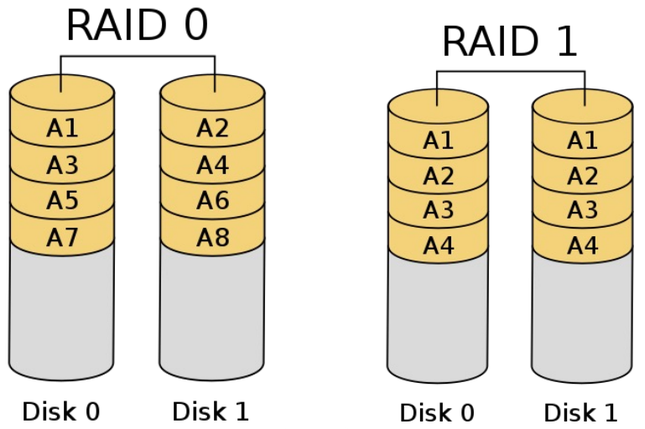
RAID: Redundant Arrays of (Inexpensive) Disks

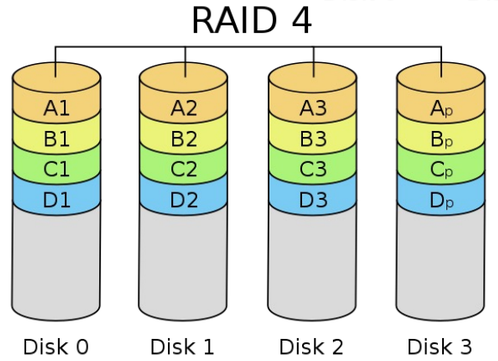
- **Data is stored across multiple disks**
- **Files are "striped" across multiple disks**
- **Redundancy yields high data availability**
  - Availability: service still provided to user, even if some components failed
- **Disks will still fail**
- **Contents reconstructed from data redundantly stored in the array**
  - Capacity penalty to store redundant info
  - Bandwidth penalty to update redundant info

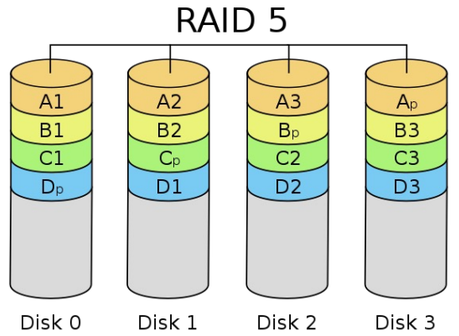
CC BY NC SA

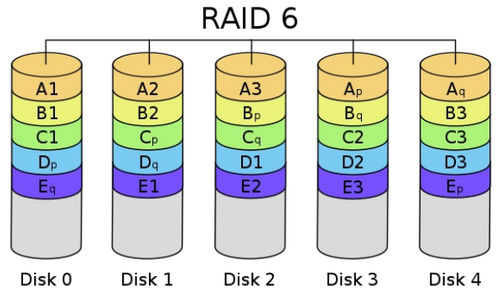
RAID levels

- **RAID 0: Stripe data across 2+ disks**
  No redundancy, disk failure = lost data!
- **RAID 1: Mirror the disk**
- **RAID 4: Parity disk block-level striping**
- **RAID 5: Block-level striping, but distribute the parity across all disks**
- **RAID 6: Have another parity block, and also distribute parity across all disks**
  Can support *two* concurrent disk failures











"And in Conclusion…"

- **Great Idea: Redundancy to Get Dependability**
  - Spatial (extra hardware) and Temporal (retry if error)
- **Reliability: MTTF**
- **Availability: % uptime (MTTF/MTTF+MTTR)**
- **Memory**
  - Hamming distance 2: Parity for Single Error Detect
  - Hamming distance 3: Single Error Correction Code + encode bit position of error
- **Treat disks like memory, except you know when a disk has failed — erasure makes parity an Error Correcting Code**
- **RAID-0, -1, -4, -5, -6: Interleaved data and parity**

cc BY NC SA